



Arrays

Aula 05 · Bloco 2 — Estruturas de Dados

Guardar, acessar e percorrer coleções de valores

Nesta aula

1. O problema que o array resolve
2. Índices, `length` e o **elemento final**
3. Os métodos essenciais
4. **Três formas** de percorrer
5. Os padrões clássicos que resolvem metade dos problemas

O problema

Como guardar as notas de 40 alunos? Com `nota1`, `nota2` ... `nota40` ?

Não. Para coleções, **array** — uma lista ordenada:

```
const notas = [8.5, 7.0, 9.2, 6.8, 10.0];  
const nomes = ["Ana", "Bruno", "Carla"];  
const mistura = [42, "texto", true];    // pode misturar tipos – ≠ Java!  
const vazio = [];
```

O índice começa em zero

```
const frutas = ["maçã", "banana", "uva", "manga"];  
//  índices:      0      1      2      3  
  
console.log(frutas[0]);           // maçã  
console.log(frutas[10]);          // undefined – não explode!  
console.log(frutas.length);       // 4  
console.log(frutas[frutas.length - 1]); // manga ← decore este padrão
```

⚠️ Acessar índice que não existe devolve **undefined** em vez de dar erro. Silencioso — e por isso perigoso.

🤔 Espera: **frutas** não é **const** ?

```
const frutas = ["maçã", "banana"];  
frutas[1] = "morango";      // ✅ funciona!  
frutas = outroArray;        // ❌ TypeError
```

O **const** impede **trocar a caixa**.

Não impede **mexer dentro dela**.

Adicionar e remover

```
const fila = ["Ana", "Bruno"];

fila.push("Carla");      // adiciona no FIM
fila.pop();              // remove do FIM
fila.unshift("Zeca");    // adiciona no INÍCIO
fila.shift();            // remove do INÍCIO
```

💡 **Mnemônico:** `push` / `pop` trabalham no fim — são os mais usados. `shift` / `unshift` mexem no início e são mais caros, porque reposicionam todo o resto.

Procurar, fatiar e juntar

```
const nums = [10, 20, 30, 20];  
console.log(nums.includes(30)); // true - existe?  
console.log(nums.indexOf(20)); // 1 - posição da 1ª  
console.log(nums.indexOf(99)); // -1 - não existe  
  
const letras = ["a", "b", "c", "d"];  
console.log(letras.slice(1, 3)); // ["b","c"] - do 1 ATÉ o 3 (exclusivo)  
console.log(letras.join(" - ")); // "a - b - c - d" - vira string  
console.log("1,2,3".split(",")); // ["1","2","3"] - o caminho inverso
```

😱 A pegadinha do `sort()`

```
const nums = [10, 9, 100, 2];  
nums.sort();  
// [10, 100, 2, 9] ← ordenou como TEXT0!
```

O `sort()` sem argumento compara **strings**.

```
nums.sort((a, b) => a - b); // ✅ [2, 9, 10, 100]
```

Para número, sempre passe a função de comparação.

Três formas de percorrer

```
const notas = [8.5, 7.0, 9.2];

for (let i = 0; i < notas.length; i++) {           // quando PRECISA do índice
  console.log(`Aluno ${i + 1}: ${notas[i]}`);
}

for (const nota of notas) {                         // quando só importa o VALOR
  console.log(nota);
}

notas.forEach((nota) => console.log(nota));         // estilo funcional – aula 08
```

Os três padrões clássicos

```
let soma = 0; // SOMAR / MÉDIA
for (const n of notas) soma += n;
console.log((soma / notas.length).toFixed(2));

let maior = notas[0]; // MAIOR VALOR
for (const n of notas) if (n > maior) maior = n;

let aprovados = 0; // CONTAR COM CONDIÇÃO
for (const n of notas) if (n >= 7) aprovados++;
```

Decore os três esqueletos — eles resolvem metade dos problemas com listas.

Exercícios da aula

Na pasta `aula-05/` :

1. `ex01.js` — 5 favoritos: primeiro, último, total e lista numerada;
2. `ex02.js` — semana de temperaturas: média, maior, menor, dias acima da média;
3. `ex03.js` — fila de atendimento com `push` e `shift` ;
4. `ex04.js` — `inverter(array)` **sem** usar `.reverse()` ;
5. `ex05.js` — `contar0correncias(array, valor)` ;
6. **Desafio** 🌶️ `ex06.js` — remover duplicatas só com `for` , `includes` e `push` .

Próxima aula

Aula 06 — Objetos

O array guarda uma lista de valores.

O objeto representa **uma coisa**, com atributos.